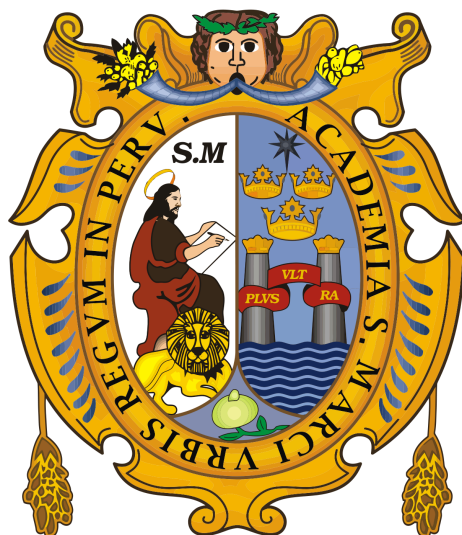


«Año de la Esperanza y el Fortalecimiento de la Democracia»

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



CURSO:

Redes de Computadora

DOCENTE:

Hernan Oswaldo Villafuerte Barreto

INTEGRANTES:

Carlos Nicolas Morales Cuellar (23190329)

Ramos Calzada Jose Martin (23190341)

Ronald Enrique Rojas Crisostomo (23190346)

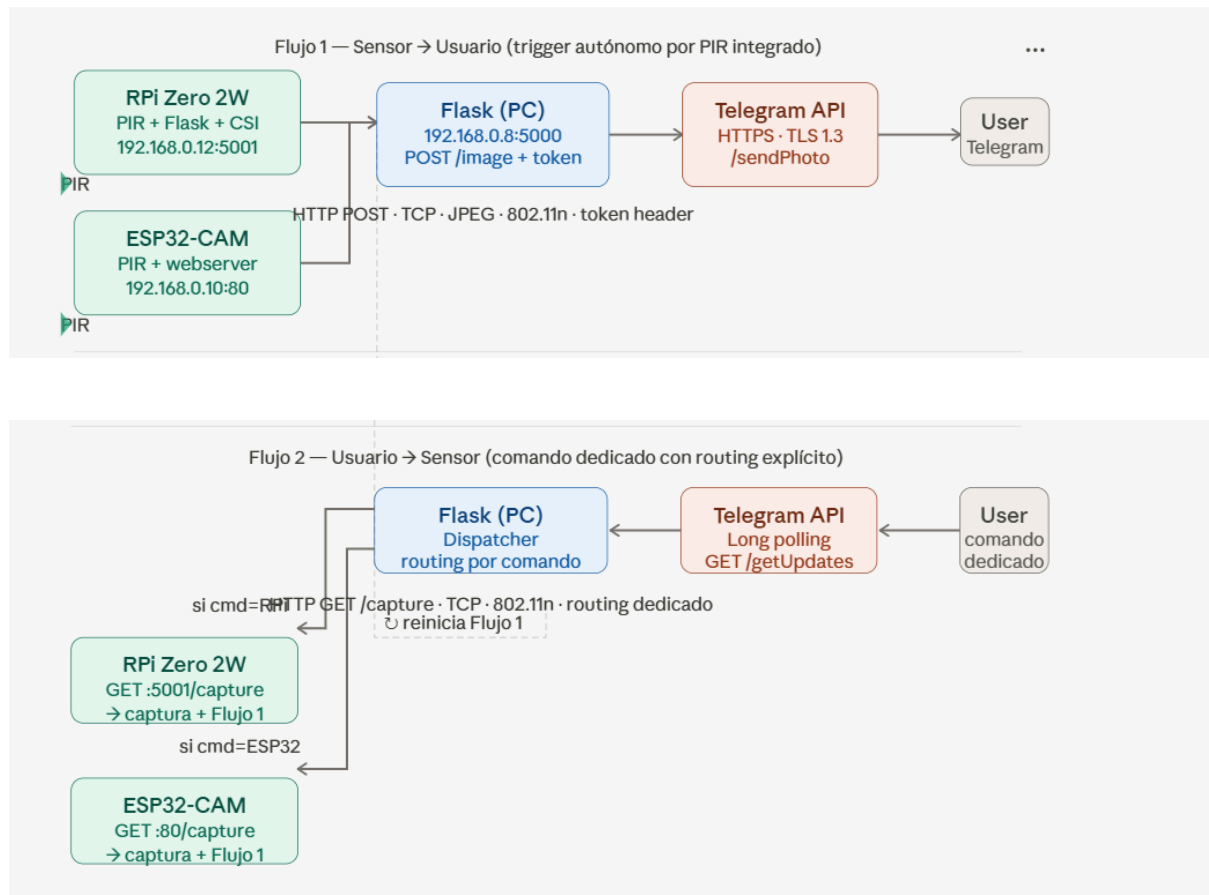
Perú - Lima

2026

Planteamiento Inicial del Proyecto de Investigación

Análisis del tráfico en una arquitectura IoT bilateral mediante Wireshark

Flujo de Trabajo del Sistema IOT



2. Problema de investigación

La arquitectura IoT bilateral implementada mediante sensor PIR, Raspberry Pi Zero 2W, servidor Flask y la API de Telegram permite automatizar la captura y envío de información en ambos sentidos de comunicación. Sin embargo, no existe visibilidad detallada del tráfico generado entre estos componentes, especialmente en las comunicaciones HTTP internas y HTTPS externas. Esta falta de observabilidad limita el análisis de cabeceras, puertos, tiempos de respuesta, secuencias TCP y mecanismos de autenticación como tokens Bearer, dificultando la evaluación del desempeño y seguridad del sistema. Por ello surge la necesidad de analizar y caracterizar el tráfico de red generado por esta arquitectura utilizando Wireshark, a fin de identificar parámetros de comunicación relevantes y posibles debilidades en la red.

3. Justificación

La investigación es pertinente porque permite aplicar conocimientos teóricos de redes en un entorno IoT real, integrando análisis práctico de protocolos, puertos, cabeceras y mecanismos de autenticación. El uso de Wireshark proporciona evidencia empírica sobre cómo interactúan los dispositivos y servicios dentro de la arquitectura propuesta, permitiendo evaluar desempeño, latencia y exposición de metadatos. Además, el estudio fortalece la comprensión de la seguridad en sistemas IoT híbridos y genera una base técnica para futuras mejoras en robustez y protección del tráfico, constituyendo un aporte académico y práctico para el análisis de redes en aplicaciones IoT.

4. Objetivo general

Analizar y caracterizar el tráfico de red generado en una arquitectura IoT bilateral basada en Raspberry Pi, Flask y Telegram utilizando Wireshark, para identificar parámetros de comunicación, mecanismos de autenticación y posibles vulnerabilidades en la interacción entre dispositivos y servicios.

5. Objetivos específicos

1. Capturar el tráfico de red generado entre Raspberry Pi, servidor Flask y API de Telegram mediante Wireshark.
2. Analizar cabeceras, puertos, protocolos y tiempos de respuesta presentes en los flujos de comunicación.
3. Identificar el comportamiento de los mecanismos de autenticación y transporte utilizados en la arquitectura.
4. Evaluar posibles debilidades de red y oportunidades de mejora en seguridad y desempeño del sistema IoT.

6. Hipótesis

El análisis del tráfico de red mediante Wireshark permitirá identificar parámetros críticos de comunicación, evidenciar el comportamiento de los mecanismos de autenticación y detectar posibles debilidades en la arquitectura IoT bilateral, contribuyendo a mejorar la comprensión del desempeño y la seguridad del sistema.

7. Variables de análisis en Wireshark

Las variables a analizar incluyen protocolos utilizados (HTTP, HTTPS, TCP), puertos de servicio (8888 y 443), cabeceras HTTP, tiempos de respuesta, tamaño de paquetes, secuencias TCP, flags de control, tokens de autenticación visibles en tráfico no cifrado y metadatos asociados a sesiones TLS. Estas variables permitirán caracterizar la comunicación entre nodos y evaluar eficiencia y seguridad.

8. Metodología experimental

La metodología consistirá en instrumentar la arquitectura IoT propuesta y capturar tráfico en puntos estratégicos de la red usando Wireshark, principalmente en la comunicación entre Raspberry Pi y servidor Flask y en la interacción del servidor con Telegram. Posteriormente se clasificarán los paquetes por protocolo y flujo, se analizarán cabeceras, puertos, latencias y mecanismos de autenticación, y finalmente se interpretarán los resultados para identificar patrones de comunicación y posibles debilidades en la red.

9. Métricas iniciales propuestas (estimaciones)

- **Número esperado de dispositivos: 3 a 4 nodos de red principales.** Estos incluyen la Raspberry Pi Zero 2W (nodo de percepción/transporte), el Servidor Flask (si se despliega en un equipo separado dentro de la LAN (en nuestro caso una laptop)), el enrutador/gateway y el dispositivo final del usuario interactuando con Telegram (ej. celular). (Nota: El sensor PIR interactúa a nivel de hardware por pines GPIO, por lo que no cuenta como un dispositivo IP independiente).
- **VLANs requeridas (mínimo 2):** En consulta / Por definir. Actualmente está en duda y se validará el día de hoy con el docente si es estrictamente necesario implementar segmentación lógica mediante VLANs para este escenario. De ser un requisito obligatorio, se propondría una VLAN para el tráfico IoT (Raspberry y Servidor) y otra VLAN para la red de usuarios/gestión.
- **Subredes IPv4 con subneteo justificado:** Asumiendo que se implemente una segmentación para el entorno de pruebas, se propone utilizar la subred 192.168.10.0/28 para la capa de procesamiento y percepción IoT. Justificación: Una máscara /28 proporciona hasta 14 direcciones IP útiles. Al ser una arquitectura con pocos dispositivos, esta máscara es ideal para contener el dominio de broadcast, evitar el desperdicio de direcciones IP y reducir el "ruido" de red generado por otros dispositivos domésticos o del laboratorio, facilitando una captura de paquetes mucho más limpia en Wireshark.
- **Latencia máxima tolerable:** < 100 ms para la comunicación interna en la LAN (entre la Raspberry Pi y el Servidor Flask al dispararse el trigger digital) y < 2000 ms (2 segundos) para el ciclo de comunicación HTTPS externo con la API de Telegram. Esta latencia asegura que la alerta y la imagen lleguen al usuario en un tiempo prudente para considerarse un sistema de monitoreo efectivo.
- **Throughput mínimo requerido(es la cantidad real de datos útiles que se transmiten con éxito desde un origen hasta un destino en un período de tiempo determinado. Se mide normalmente en bits por segundo (bps, Kbps, Mbps):** 2 a 5 Mbps de ancho de banda de subida (uplink). Justificación: Los flujos de control (mensajes de texto de Telegram y polling) consumen pocos kilobytes. Sin embargo, el esquema muestra un flujo de HTTP POST /al_image y "respuesta con imagen". Para transmitir fotografías o capturas ligeras sin retrasos significativos en la capa de aplicación, un throughput de 2-5 Mbps garantiza una transferencia fluida y evita la congestión en el enrutador de salida.

10. Preguntas de investigación

- **Pregunta 1:** ¿Cuáles son las características principales y el comportamiento de las cabeceras, puertos, protocolos y tiempos de respuesta presentes en los flujos de comunicación de la arquitectura IoT bilateral?

- **Pregunta 2:** ¿De qué manera operan y qué nivel de exposición presentan los mecanismos de autenticación (como los tokens Bearer) y de transporte al interactuar entre la Raspberry Pi, el servidor Flask y la API de Telegram?

Observación: Token Bearer es un tipo de credencial de acceso utilizada comúnmente en la autenticación de APIs y aplicaciones web. La palabra "Bearer" significa literalmente "portador". Su principio fundamental es simple: quienquiera que posea (o "porte") el token, tiene los permisos de acceso que este otorga, sin necesidad de demostrar su identidad de ninguna otra manera.

- **Pregunta 3:** ¿Qué debilidades a nivel de red se pueden identificar mediante la captura de paquetes para determinar oportunidades de mejora en la seguridad y el desempeño general del sistema IoT?

11. Tecnología emergente propuesta

La tecnología emergente central de este proyecto es IoT (Internet of Things), complementada con un enfoque de Edge Computing ligero, dado que el procesamiento de la captura (sensor PIR) y la comunicación inicial (envío de la imagen) se realizan localmente en el borde de la red mediante la Raspberry Pi Zero 2W antes de interactuar con servicios en la nube (Telegram API).

12. Estado del arte inicial (mínimo 3 referencias)

Referencia 1: Análisis de tráfico y caracterización de protocolos en IoT

- **Contexto:** La investigación se basa en la necesidad de visibilidad detallada del tráfico en comunicaciones HTTP internas y HTTPS externas.
- **Aporte:** Estudios previos sobre la caracterización de tráfico mediante Wireshark demuestran que la inspección profunda de paquetes (DPI) es esencial para identificar patrones de latencia y anomalías en dispositivos de recursos limitados, como la Raspberry Pi Zero 2W. Esto permite validar si los tiempos de respuesta se mantienen bajo los 100 ms en la LAN, tal como se estima en el plan experimental.

Referencia 2: Seguridad en la integración de APIs de mensajería y tokens Bearer

- **Contexto:** El proyecto utiliza la Telegram API y mecanismos de autenticación mediante Tokens Bearer.
- **Aporte:** El estado del arte en seguridad para "Messaging Bots" en IoT indica que, aunque el tráfico HTTPS está cifrado (puerto 443), la exposición de metadatos o la persistencia de tokens en la capa de aplicación (servidor Flask) puede representar una vulnerabilidad. Esta referencia sustenta la pregunta de investigación sobre el nivel de exposición de los mecanismos de transporte.

Referencia 3: Segmentación lógica y VLANs en entornos de Edge Computing

- **Contexto:** Se propone el uso de una subred 192.168.10.0/28 y la posible implementación de VLANs para separar el tráfico IoT del tráfico de gestión.
- **Aporte:** La literatura técnica actual en redes industriales señala que la segmentación lógica mediante el estándar **IEEE 802.1Q (VLANs)** es la mejor práctica para contener dominios de broadcast y reducir el "ruido" de red. Esto es crítico para obtener capturas de paquetes limpias en Wireshark y proteger la red principal de posibles debilidades en el nodo de percepción (sensor PIR).

13. Marco Teórico Aplicado al Escenario IoT

Modelo OSI — Mapeo por nodo

► Nodo 1 — Raspberry Pi Zero 2W (192.168.0.12:5001 · Flask + CSI)		
Capa	Función del Sistema	Protocolos
7 — Aplicación	Flujo 1: Servidor Flask RPI expone GET :5001/capture. Captura imagen vía libcamera/picamera2, construye HTTP POST multipart/form-data con JPEG + token hacia 192.168.0.8:5000/image. Flujo 2: Responde GET /capture enviado por Flask-PC; retorna imagen y desencadena Flujo 1.	HTTP/1.1, REST, multipart/form-data
6 — Presentación	Compresión JPEG de frame CSI mediante pipeline libcamera (YUV→JPEG). Codificación en CPU ARM Cortex-A53.	JPEG (ISO/IEC 10918)
5 — Sesión	Sesiones HTTP cortas (request/response). Sin estado entre capturas.	HTTP/1.1 Keep-Alive (opcional)
4 — Transporte	TCP garantiza entrega ordenada del JPEG. Three-way handshake por sesión hacia 192.168.0.8:5000.	TCP (RFC 793), puerto destino 5000
3 — Red	Origen IP fijo 192.168.0.12. Destino: 192.168.0.8 (Flask-PC, mismo segmento). Sin gateway.	IPv4 (RFC 791)
2 — Enlace	Cliente WLAN 802.11n. Tramas 802.11 data encapsulan frames Ethernet hacia el AP.	IEEE 802.11n, CSMA/CA, ARP
1 — Física	Radio 2.4 GHz OFDM (802.11n PHY). Bus MIPI CSI-2 (ribbon FPC) hacia módulo cámara.	IEEE 802.11n PHY, MIPI CSI-2

► Nodo 3 — ESP32-CAM (192.168.0.10:80 · webserver propio)		
Capa	Función del Sistema	Protocolos

7 — Aplicación	Flujo 1: Webserver ESP-IDF expone GET /capture. Genera HTTP POST multipart/form-data con JPEG OV2640 hacia 192.168.0.8:5000/image + token. Flujo 2: Responde GET /capture de Flask-PC; retorna JPEG y dispara Flujo 1.	HTTP/1.1, REST, multipart/form-data
6 — Presentación	OV2640 genera JPEG comprimido por hardware interno. No hay procesamiento adicional en CPU.	JPEG (hardware OV2640)
5 — Sesión	Sesiones HTTP cortas. Sin estado entre capturas. PIR como único trigger local.	HTTP/1.1
4 — Transporte	lwIP 2.x gestiona TCP. Handshake hacia 192.168.0.8:5000 por cada captura.	TCP/lwIP (RFC 793), puerto destino 5000
3 — Red	IP fija 192.168.0.10. Segmento /24 local. Sin gateway para tráfico LAN.	IPv4/lwIP (RFC 791)
2 — Enlace	Cliente WLAN 802.11n integrado (radio Xtensa LX6). AP bridge.	IEEE 802.11n, CSMA/CA
1 — Física	Radio 2.4 GHz OFDM. Bus DVP/I2S interno entre OV2640 y SoC ESP32.	IEEE 802.11n PHY, DVP (Digital Video Port)

► Nodo 4 — Flask-PC (192.168.0.8:5000 · dispatcher central)

7 — Aplicación	Hub de aplicación: (a) Flujo 1: recibe POST /image + token, valida, reenvía JPEG a Telegram POST /sendPhoto HTTPS. (b) Flujo 2: long polling GET /getUpdates; parsea JSON Telegram; routing por message.text → GET /capture a RPi:5001 o ESP32:80.	HTTP/1.1 (LAN), HTTPS/TLS 1.3 (Internet), REST, Telegram Bot API, JSON, multipart/form-data
6 — Presentación	Flujo 1: Recibe JPEG binario en POST /image; lo reenvía sin transcodificación a /sendPhoto. Flujo 2: Parsea JSON UTF-8 de /getUpdates para extracción de comandos.	JPEG (pass-through), JSON/UTF-8, TLS 1.3
5 — Sesión	Mantiene sesión TCP semi-persistente hacia api.telegram.org (long polling). Sesiones cortas y stateless hacia sensores LAN.	HTTP Keep-Alive, TLS session resumption, long polling
4 — Transporte	TCP hacia sensores LAN (puertos 5001/80) y hacia Telegram (443). Mínimo 2 handshakes TCP por ciclo de captura.	TCP (RFC 793), puertos 80, 443, 5000, 5001
3 — Red	IP 192.168.0.8. Enrutamiento LAN directo. Tráfico externo vía gateway NAT (0.0.0.0/0).	IPv4 (RFC 791), NAT/PAT (gateway)
2 — Enlace	NIC Wi-Fi. Cliente WLAN	IEEE 802.11n CSMA/CA
1 — Física	NIC física del PC (Wi-Fi 2.4/5 GHz).	IEEE 802.11n PHY

Modelo TCP/IP — Mapeo por nodo

► Nodo 2 — RPi Zero 2W (192.168.0.12)		
Aplicación(≡ L5-L7)	Flask server :5001 responde GET /capture (Flujo 2). Cliente HTTP: POST multipart/form-data JPEG+token a 192.168.0.8:5000/image (Flujo 1). Captura CSI via libcamera.	HTTP/1.1, REST, multipart/form-data, JPEG
Transporte(≡ L4)	TCP kernel Linux hacia 192.168.0.8:5000. Entrega ordenada del payload JPEG.	TCP (RFC 793)
Internet(≡ L3)	IPv4 origen 192.168.0.12 → destino 192.168.0.8. Sin gateway (mismo /24).	IPv4 (RFC 791), ARP
Acceso a Red(≡ L1+L2)	Cliente WLAN 802.11n (BCM43438, 2.4 GHz). MIPI CSI-2 para datos de imagen (interno).	IEEE 802.11n, CSMA/CA
► Nodo 3 — ESP32-CAM (192.168.0.10)		
Aplicación(≡ L5-L7)	Webserver ESP-IDF :80 responde GET /capture (Flujo 2). Cliente HTTP: POST multipart/form-data JPEG+token a 192.168.0.8:5000/image (Flujo 1). JPEG generado por OV2640.	HTTP/1.1, REST, multipart/form-data, JPEG
Transporte(≡ L4)	TCP lwIP 2.x hacia 192.168.0.8:5000. Gestión de ventana limitada.	TCP/lwIP (RFC 793)
Internet(≡ L3)	IPv4 origen 192.168.0.10 → destino 192.168.0.8. Sin gateway.	IPv4/lwIP (RFC 791), ARP
Acceso a Red(≡ L1+L2)	Radio Wi-Fi integrado 2.4 GHz 802.11n. DVP para transferencia OV2640→SoC.	IEEE 802.11n, CSMA/CA
► Nodo 4 — Flask-PC (192.168.0.8)		
Aplicación(≡ L5-L7)	Dispatcher central: valida token + reenvía JPEG a Telegram /sendPhoto (Flujo 1). Long polling /getUpdates + routing por comando + GET /capture a sensor (Flujo 2). Proxy de protocolo HTTP↔HTTPS.	HTTP/1.1 (LAN), HTTPS/TLS 1.3 (WAN), Telegram Bot API, JSON, multipart/form-data
Transporte(≡ L4)	TCP Linux hacia sensores (puertos 5001, 80) y Telegram (443). Long polling = 1 TCP persistente + 1 TCP efímera por captura.	TCP (RFC 793)
Internet(≡ L3)	192.168.0.8 en LAN. Tráfico Telegram vía NAT/PAT gateway. Default route → ISP → api.telegram.org.	IPv4 (RFC 791), NAT/PAT

Acceso a Red(≡ L1+L2)	NIC PC (Wi-Fi 802.11n o Ethernet 802.3). Enlace hacia AP/switch.	IEEE 802.11n
-----------------------	--	--------------

Modelo OSI y TCP/IP (evidencias)

El sistema actual se fundamenta en los estándares de los modelos OSI y TCP/IP, operando como un sistema IoT bidireccional.

- **Capa de Aplicación (OSI L7 / TCP/IP Aplicación):** Se encarga de la lógica del sistema. El nodo sensor (RPi/ESP32) genera solicitudes HTTP POST con imágenes JPEG hacia un servidor Flask, el cual actúa como proxy o "dispatcher"(es decir, un **encaminador o distribuidor de solicitudes**, su tarea es **recibir las peticiones HTTP POST** que llegan desde los nodos sensores (por ejemplo, una Raspberry Pi o un ESP32) y **redirigirlas** hacia otro destino, en este caso la **API de Telegram**). Flask traduce las peticiones HTTP locales a peticiones HTTPS hacia la API de Telegram, utilizando formatos JSON (Flask usa **formatos JSON para control**, lo que significa que los **parámetros de configuración o comandos** (por ejemplo, el texto del mensaje, el ID del chat, el token del bot) para control y *multipart/form-data* para las imágenes.
- A continuación se ha hecho uso del nmap para el escaneo de puertos, adjuntado como evidencia:

```
C:\Windows\system32>nmap -Pn -sT -sV -p- 192.168.0.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 21:24 -0500
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 75.11% done; ETC: 21:24 (0:00:05 remaining)
Stats: 0:00:23 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 21:24 (0:00:00 remaining)
Nmap scan report for 192.168.0.12
Host is up (0.0056s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Raspbian 7+deb13u2 (protocol 2.0)
5001/tcp   open  http      Werkzeug httpd 3.1.8 (Python 3.13.5)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 39.56 seconds
```

IoT. En el nodo 192.168.0.12, el servicio en el puerto 5001 es identificado como Werkzeug httpd 3.1.8 (Python 3.13.5), lo que confirma que el servidor Flask está activo y listo para actuar como dispatcher entre la LAN y la API de Telegram. Asimismo, se detecta el servicio OpenSSH 10.0p2 en el puerto 22, utilizado para la administración remota de la Raspberry Pi.

- **Capa de Presentación y Sesión (OSI L6-L5):** Los datos visuales se comprimen en formato JPEG. A nivel de sesión, el flujo LAN es *stateless* (cada captura es independiente), mientras que hacia Telegram se utiliza **long polling** (técnica de comunicación entre cliente y servidor usada cuando se necesita mantener una conexión "activa", esto significa que el sistema mantiene una conexión continua con Telegram para recibir o enviar mensajes sin interrupciones, aunque cada solicitud HTTP sea técnicamente independiente), creando una sesión TCP semi-persistente. La

seguridad TLS 1.3 (protocolo que cifra la comunicación entre dos puntos en Internet, usada en HTTPS) se aplica solo en el tramo externo (Internet), dejando el tráfico de la LAN en texto plano.

- **Capa de Transporte (OSI L4 / TCP/IP Transporte):** Se utiliza exclusivamente el protocolo TCP. Se establece una conexión persistente para el polling de Telegram y conexiones efímeras para cada captura enviada desde los sensores al servidor Flask.

```
C:\Windows\system32>nmap -sT -sV -vv -p 80,81 --max-retries 2 --reason 192.168.0.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 21:20 -0500
NSE: Loaded 48 scripts for scanning.
Initiating ARP Ping Scan at 21:20
Scanning 192.168.0.10 [1 port]
Completed ARP Ping Scan at 21:21, 0.14s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 21:21
Completed Parallel DNS resolution of 1 host. at 21:21, 0.50s elapsed
Initiating Connect Scan at 21:21
Scanning 192.168.0.10 [2 ports]
Discovered open port 80/tcp on 192.168.0.10
Discovered open port 81/tcp on 192.168.0.10
Completed Connect Scan at 21:21, 0.01s elapsed (2 total ports)
Initiating Service scan at 21:21
Scanning 2 services on 192.168.0.10
Completed Service scan at 21:21, 11.69s elapsed (2 services on 1 host)
NSE: Script scanning 192.168.0.10.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 21:21
Completed NSE at 21:21, 0.09s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 21:21
Completed NSE at 21:21, 0.04s elapsed
Nmap scan report for 192.168.0.10
Host is up, received arp-response (0.064s latency).
Scanned at 2026-05-03 21:21:00 Hora est. Pacífico, Sudamérica for 12s

PORT      STATE SERVICE REASON  VERSION
80/tcp    open  http   syn-ack
81/tcp    open  hosts2-ns? syn-ack
```

En la capa de transporte, se confirma el uso del protocolo TCP para la comunicación entre nodos. El escaneo de servicios revela que el puerto 80/tcp (HTTP) en el sensor ESP32 y el puerto 5001/tcp (Flask) en la Raspberry Pi están en estado open con la bandera syn-ack. Esto garantiza que el three-way handshake de TCP se completa exitosamente, permitiendo una transferencia de datos confiable para las imágenes JPEG y los comandos de control.

- **Capa de Red y Enlace (OSI L3-L2 / TCP/IP Internet y Acceso):** Se opera sobre una red local IPv4 plana y el enrutamiento externo se realiza mediante NAT en el gateway. El acceso al medio inalámbrico es gestionado mediante 802.11n (Wi-Fi 4) utilizando CSMA/CA (permite que los dispositivos Wi-Fi transmitan datos sin interferirse, **evitando colisiones** mediante coordinación previa, algo esencial en redes inalámbricas).

Capa de Red - Protocolo ICMP

El script implementa un dentro de la subred **192.168.0.0/24**, utilizando **mensajes ICMP Echo Request/Reply** (ping) para identificar dispositivos accesibles a nivel de red.

A nivel operativo, recorre secuencialmente el rango de direcciones IP (**192.168.0.1** a **192.168.0.254**), enviando dos solicitudes ICMP a cada host mediante la función **Test-Connection**. Este proceso corresponde a un **escaneo de red de capa 3 (Network Layer del modelo OSI)**, basado en el protocolo ICMP encapsulado sobre IP.

El script filtra exclusivamente las respuestas exitosas (**StatusCode = 0**), lo que garantiza la consideración únicamente de **hosts que responden con ICMP Echo Reply**, descartando silencios, timeouts o errores de red.

Para cada host activo, se calculan métricas relevantes de rendimiento:

- **Latencia promedio, mínima y máxima (ms)**: indican el tiempo de ida y vuelta (RTT), útil para evaluar calidad de enlace.
- **TTL (Time To Live)**: permite inferir indirectamente el tipo de sistema operativo o la cantidad de saltos en la ruta.

La salida se presenta en consola en tiempo real, mostrando únicamente dispositivos activos

```
PS C:\Users\PC - Usuario> for ($i = $start; $i -le $end; $i++) {
>>     $ip = "$network.$i"
>>
>>     # Enviar 2 solicitudes ICMP
>>     $ping = Test-Connection -ComputerName $ip -Count 2 -ErrorAction SilentlyContinue
>>
>>     # Filtrar solo respuestas exitosas (Echo Reply)
>>     $success = $ping | Where-Object { $_.StatusCode -eq 0 }
>>
>>     if ($success -and $success.Count -gt 0) {
>>         # Métricas
>>         $avg = ($success | Measure-Object -Property ResponseTime -Average).Average
>>         $min = ($success | Measure-Object -Property ResponseTime -Minimum).Minimum
>>         $max = ($success | Measure-Object -Property ResponseTime -Maximum).Maximum
>>         $ttl = $success[0].TimeToLive
>>
>>         # Salida en consola
>>         Write-Host ("[SUCCESS] {0} | Respuestas: {1}/2 | Prom: {2} ms | Min: {3} ms | Max: {4} ms | TTL: {5}" -f `
>>             $ip,
>>             $success.Count,
>>             [math]::Round($avg,2),
>>             $min,
>>             $max,
>>             $ttl
>>         )
>>     }
>> }
[SUCCESS] 192.168.0.1 | Respuestas: 2/2 | Prom: 0.5 ms | Min: 0 ms | Max: 1 ms | TTL: 80
[SUCCESS] 192.168.0.3 | Respuestas: 2/2 | Prom: 1.5 ms | Min: 1 ms | Max: 2 ms | TTL: 80
[SUCCESS] 192.168.0.5 | Respuestas: 2/2 | Prom: 35.5 ms | Min: 3 ms | Max: 68 ms | TTL: 80
[SUCCESS] 192.168.0.6 | Respuestas: 2/2 | Prom: 47 ms | Min: 16 ms | Max: 78 ms | TTL: 80
[SUCCESS] 192.168.0.7 | Respuestas: 2/2 | Prom: 514 ms | Min: 494 ms | Max: 534 ms | TTL: 80
[SUCCESS] 192.168.0.8 | Respuestas: 2/2 | Prom: 0 ms | Min: 0 ms | Max: 0 ms | TTL: 80
[SUCCESS] 192.168.0.9 | Respuestas: 2/2 | Prom: 255.5 ms | Min: 204 ms | Max: 307 ms | TTL: 80
[SUCCESS] 192.168.0.10 | Respuestas: 2/2 | Prom: 3.5 ms | Min: 2 ms | Max: 5 ms | TTL: 80
[SUCCESS] 192.168.0.11 | Respuestas: 2/2 | Prom: 49 ms | Min: 22 ms | Max: 76 ms | TTL: 80
[SUCCESS] 192.168.0.12 | Respuestas: 2/2 | Prom: 38 ms | Min: 3 ms | Max: 73 ms | TTL: 80
```

En esta capa se valida el direccionamiento físico y la resolución de nombres local. La captura muestra un escaneo de tipo ARP Ping, donde se confirma

que los nodos responden mediante el protocolo ARP (received arp-response). Esto permite obtener la Dirección MAC de cada dispositivo, identificando al fabricante de las interfaces: **Espressif** para el nodo sensor (192.168.0.10) y **Raspberry Pi Trading** para el nodo de control (192.168.0.12).

```
C:\Windows\system32>nmap -sn -PR -vv --max-retries 2 --reason 192.168.0.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 21:08 -0500
Initiating ARP Ping Scan at 21:08
Scanning 255 hosts [1 port/host]
Completed ARP Ping Scan at 21:08, 4.43s elapsed (255 total hosts)
Initiating Parallel DNS resolution of 10 hosts. at 21:08
Completed Parallel DNS resolution of 10 hosts. at 21:08, 0.51s elapsed
Nmap scan report for 192.168.0.0 [host down, received no-response]
Nmap scan report for 192.168.0.1
Host is up, received arp-response (0.0070s latency).
MAC Address: A0:39:EE:96:F6:FC (Sagemcom Broadband SAS)
Nmap scan report for 192.168.0.2
Host is up, received arp-response (0.0070s latency).
MAC Address: A0:39:EE:96:F6:FD (Sagemcom Broadband SAS)
Nmap scan report for 192.168.0.3
Host is up, received arp-response (0.041s latency).
MAC Address: 8A:04:F9:3E:31:4E (Unknown)
Nmap scan report for 192.168.0.4 [host down, received no-response]
Nmap scan report for 192.168.0.5
Host is up, received arp-response (0.062s latency).
MAC Address: EE:39:18:04:ED:90 (Unknown)
Nmap scan report for 192.168.0.6
Host is up, received arp-response (0.049s latency).
MAC Address: 96:07:30:F6:37:F4 (Unknown)
Nmap scan report for 192.168.0.7
Host is up, received arp-response (0.13s latency).
MAC Address: C6:09:97:87:23:B6 (Unknown)
Nmap scan report for 192.168.0.9
Host is up, received arp-response (0.040s latency).
MAC Address: 7E:0E:26:79:55:94 (Unknown)
Nmap scan report for 192.168.0.10
Host is up, received arp-response (0.069s latency).
MAC Address: 30:C6:F7:05:0C:7C (Espressif)
Nmap scan report for 192.168.0.11
Host is up, received arp-response (0.048s latency).
MAC Address: 22:FE:C7:11:EB:91 (Unknown)
Nmap scan report for 192.168.0.12
Host is up, received arp-response (0.041s latency).
MAC Address: 88:A2:9E:5A:FC:5E (Raspberry Pi (Trading))
Nmap scan report for 192.168.0.13 [host down, received no-response]
```

Ahora con respecto a la capa de red, las evidencias son las siguientes:

IP DE LA PC:

```
C:\Windows\system32>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet:

    Sufijo DNS específico para la conexión. . . :
    Dirección IPv6 . . . . . : 2800:200:e800:3a9a:cd3a:bc3b:2bf9:be35
    Dirección IPv6 temporal. . . . . : 2800:200:e800:3a9a:4cf5:5bb7:ce22:6e3f
    Vínculo: dirección IPv6 local. . . : fe80::14ff:9d39:7177:3283%18
    Dirección IPv4. . . . . : 192.168.0.8
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . : fe80::a239:eeff:fe96:f6fc%18
                                           192.168.0.1
```

IP DEL RASPBERRY:

```
pi@raspberrypi:~$ ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 17 bytes 2722 (2.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 17 bytes 2722 (2.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.12 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::8aa2:9eff:fe5a:fc5e prefixlen 64 scopeid 0x20<link>
    inet6 2800:200:e800:3a9a:8aa2:9eff:fe5a:fc5e prefixlen 64 scopeid 0x0<global>
    ether 88:a2:9e:5a:fc:5e txqueuelen 1000 (Ethernet)
    RX packets 73415 bytes 4504399 (4.2 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 69042 bytes 3814584 (3.6 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Se verifica el direccionamiento lógico IPv4 bajo el esquema de subred 192.168.0.0/24. El comando ipconfig confirma la IP del PC de despacho como 192.168.0.8, mientras que ifconfig valida la IP de la Raspberry Pi como 192.168.0.12.

Adicionalmente, el análisis del protocolo ICMP mediante el script de PowerShell demuestra la conectividad de capa 3. Se observa un TTL de 80 constante, lo que indica que los paquetes no atraviesan routers intermedios (salto 0), y métricas de latencia estables, excepto en el nodo 192.168.0.9, que presenta un promedio de 255.5 ms, sugiriendo congestión o debilidad en la señal inalámbrica en ese punto específico.

Capa de Enlace: VLAN, Spanning Tree y Agregación

- **VLAN (Virtual LAN - 802.1Q):** Permiten segmentar lógicamente una red física. Aplicación al caso: Actualmente, el sistema opera en un segmento plano (192.168.0.0/24) sin VLANs. Esto significa que todos los nodos comparten el mismo dominio de broadcast. Quizás al implementar una VLAN exclusiva para los dispositivos IoT mejoraría el rendimiento y limitaría la superficie de ataque.
- **Spanning Tree Protocol (STP/RSTP):** Previene bucles de capa 2 en topologías redundantes. Aplicación al caso: Al ser una red inalámbrica simple conectada a un único Access Point (AP) que actúa como bridge L2, no existen bucles físicos.
- **Agregación de Enlaces (LACP):** Combina múltiples enlaces físicos en uno lógico para aumentar el ancho de banda. No se aplica actualmente, ya que el medio es enteramente inalámbrico (RF 2.4 GHz) para los nodos finales.

Capa de red: Direccionamiento y Enrutamiento

- **IPv4 vs IPv6:** El sistema utiliza exclusivamente IPv4 bajo el estándar RFC 1918 (direcciones privadas). No hay presencia de IPv6 activo.
- **Subneteo (CIDR/VLSM):** La red actual es un único bloque /24 (192.168.0.0/24), sin subredes internas.
- **Enrutamiento Estático/Dinámico:** El tráfico interno se resuelve directamente en capa 2 mediante resolución ARP sin pasar por el gateway. Para el tráfico externo (Telegram), se utiliza una ruta por defecto (0.0.0.0/0) hacia el router del ISP. No se usan protocolos dinámicos como OSPF o BGP.

Análisis de protocolos y latencia

Protocolo ARP:

```
C:\Windows\system32>arp -a

Interfaz: 192.168.56.1 --- 0xc
Dirección de Internet      Dirección física      Tipo
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático

Interfaz: 192.168.0.8 --- 0x12
Dirección de Internet      Dirección física      Tipo
192.168.0.1                a0-39-ee-96-f6-fc    dinámico
192.168.0.12               88-a2-9e-5a-fc-5e    dinámico
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
```

El análisis del protocolo Address Resolution Protocol se realizó mediante la inspección de la tabla ARP del sistema operativo durante la ejecución del sistema IoT. Se observó la presencia de entradas dinámicas correspondientes a la Raspberry Pi (192.168.0.12), el router (192.168.0.1) y el propio equipo local (192.168.0.8). Estas entradas indican que el sistema ha realizado correctamente la resolución de direcciones IP a direcciones MAC dentro de la red local. Este proceso es fundamental para habilitar la comunicación entre el servidor Flask y la Raspberry Pi antes del establecimiento de conexiones basadas en TCP/IP.

Como se vería usando Wireshark:

Capturing from Ethernet

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

arp

No.	Time	Source	Destination	Protocol	Length
71	1.800371800	0a:e0:af:dc:0a:bf	Broadcast	ARP	
72	1.882177900	RaspberryPi_5a:fc:5e	0a:e0:af:dc:0a:bf	ARP	
207	6.354767900	Espressif_05:0c:7c	Broadcast	ARP	
889	44.536550800	0a:e0:af:dc:0a:bf	SagemcomBroa_96:f6:...	ARP	
890	44.537028700	SagemcomBroa_96:f6:...	0a:e0:af:dc:0a:bf	ARP	

Length	Info
42	Who has 192.168.0.12? Tell 192.168.0.8
60	192.168.0.12 is at 88:a2:9e:5a:fc:5e
60	ARP Announcement for 192.168.0.10
42	Who has 192.168.0.1? Tell 192.168.0.8
60	192.168.0.1 is at a0:39:ee:96:f6:fc

C:\Windows\System32\netstat -ano

Conexiones activas

Proto	Dirección local	Dirección remota	Estado	PID
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	424
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:3580	0.0.0.0:0	LISTENING	3956
TCP	0.0.0.0:5000	0.0.0.0:0	LISTENING	9592
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	3796
TCP	0.0.0.0:5357	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:7680	0.0.0.0:0	LISTENING	3548
TCP	0.0.0.0:49664	0.0.0.0:0	LISTENING	808
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING	720
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING	1300
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	1612
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING	3192
TCP	0.0.0.0:49671	0.0.0.0:0	LISTENING	702
TCP	0.0.0.0:50065	0.0.0.0:0	LISTENING	4996
TCP	0.0.0.0:59110	0.0.0.0:0	LISTENING	5144
TCP	0.0.0.0:59111	0.0.0.0:0	LISTENING	5228
TCP	127.0.0.1:2680	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:5939	0.0.0.0:0	LISTENING	3480
TCP	127.0.0.1:27182	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:49672	127.0.0.1:49673	ESTABLISHED	5228
TCP	127.0.0.1:49673	127.0.0.1:49672	ESTABLISHED	5228
TCP	127.0.0.1:49674	127.0.0.1:49675	ESTABLISHED	5136
TCP	127.0.0.1:49675	127.0.0.1:49674	ESTABLISHED	5136
TCP	127.0.0.1:49676	127.0.0.1:49677	ESTABLISHED	5144
TCP	127.0.0.1:49677	127.0.0.1:49676	ESTABLISHED	5144
TCP	127.0.0.1:49678	0.0.0.0:0	LISTENING	5124
TCP	127.0.0.1:49678	127.0.0.1:49683	ESTABLISHED	5124
TCP	127.0.0.1:49683	127.0.0.1:49678	ESTABLISHED	3740
TCP	127.0.0.1:49789	0.0.0.0:0	LISTENING	13516
TCP	127.0.0.1:50034	0.0.0.0:0	LISTENING	4996
TCP	127.0.0.1:50034	127.0.0.1:50048	ESTABLISHED	4996
TCP	127.0.0.1:50035	0.0.0.0:0	LISTENING	12628
TCP	127.0.0.1:50035	127.0.0.1:50036	ESTABLISHED	12628
TCP	127.0.0.1:50036	127.0.0.1:50035	ESTABLISHED	4996
TCP	127.0.0.1:50048	127.0.0.1:50034	ESTABLISHED	4704
TCP	127.0.0.1:50067	127.0.0.1:50068	ESTABLISHED	12628
TCP	127.0.0.1:50068	127.0.0.1:50067	ESTABLISHED	1544
TCP	127.0.0.1:50237	127.0.0.1:5000	TIME_WAIT	0
TCP	127.0.0.1:55878	0.0.0.0:0	LISTENING	10636
TCP	192.168.0.8:139	0.0.0.0:0	LISTENING	4
TCP	192.168.0.8:49788	2.23.232.59:443	CLOSE_WAIT	13516
TCP	192.168.0.8:50024	192.168.0.12:22	ESTABLISHED	3620
TCP	192.168.0.8:50072	149.154.175.53:443	ESTABLISHED	5224
TCP	192.168.0.8:50236	149.154.166.110:443	TIME_WAIT	0
TCP	192.168.0.8:50240	149.154.175.52:443	ESTABLISHED	5224
TCP	192.168.0.8:50242	149.154.175.53:80	TIME_WAIT	0
TCP	192.168.0.8:50243	149.154.175.52:80	TIME_WAIT	0
TCP	192.168.56.1:139	0.0.0.0:0	LISTENING	4

El protocolo Transmission Control Protocol fue analizado mediante el comando **netstat -ano**, el cual permite observar todas las conexiones activas del sistema operativo. Se identificaron puertos en estado LISTENING correspondientes al servidor Flask del sistema IoT (puerto 5000), lo que indica disponibilidad para recibir solicitudes HTTP. Asimismo, se identificaron conexiones activas entre el equipo local y la Raspberry Pi en estado ESTABLISHED, específicamente en el puerto 22 correspondiente a SSH. Esto

evidencia la existencia de comunicación TCP estable dentro del sistema IoT implementado. A su vez se observaron múltiples conexiones locales (127.0.0.1), que corresponden a procesos internos del sistema operativo y no afectan la comunicación externa del proyecto. Los servicios del sistema Windows utilizan TCP para operaciones de red internas, pero no forman parte del sistema IoT implementado.

Vista en Wireshark, para poder visualizar el envío y recibimiento de los paquetes lo importante es saber IP origen, puerto origen, IP destino, puerto destino, sabiendo eso se puede localizar

3824	39.585591800	192.168.0.12	192.168.0.8	TCP
3825	39.585809400	192.168.0.12	192.168.0.8	TCP
3826	39.595376100	192.168.0.12	192.168.0.8	HTTP
3827	39.595417200	192.168.0.8	192.168.0.12	TCP
3828	39.595467500	192.168.0.8	192.168.0.12	TCP
3829	39.597596500	192.168.0.12	192.168.0.8	TCP
226 5001 → 50423 [PSH, ACK] Seq=1 Ack=156 Win=64128 Len=172 [TCP PDU reassembled in 3826]				
60	53130 → 5000	[ACK] Seq=4471 Ack=177 Win=64128 Len=0		
60	HTTP/1.1	200 OK (text/html)		
54	50423 → 5001	[ACK] Seq=156 Ack=176 Win=262400 Len=0		
54	50423 → 5001	[FIN, ACK] Seq=156 Ack=176 Win=262400 Len=0		
60	5001 → 50423	[ACK] Seq=176 Ack=157 Win=64128 Len=0		
54	50024 → 22	[ACK] Seq=1 Ack=273 Win=1025 Len=0		

Protocolo DHCP En la Pc con el servidor flask:

```
C:\Windows\system32>ipconfig /all

Configuración IP de Windows

Nombre de host. . . . . : DESKTOP-T9FSB7E
Sufijo DNS principal. . . . . :
Tipo de nodo. . . . . : híbrido
Enrutamiento IP habilitado. . . . . : no
Proxy WINS habilitado. . . . . : no

Adaptador de Ethernet Ethernet:

Sufijo DNS específico para la conexión. . :
Descripción. . . . . : Realtek PCIe GbE Family Controller
Dirección física. . . . . : 0A-E0-AF-DC-0A-BF
DHCP habilitado. . . . . : sí
Configuración automática habilitada. . . : sí
Dirección IPv6. . . . . : 2800:200:e800:3a9a::1(Preferido)
Concesión obtenida. . . . . : lunes, 4 de mayo de 2026 09:20:12
La concesión expira. . . . . : domingo, 18 de octubre de 2026 16:26:08
Dirección IPv6. . . . . : 2800:200:e800:3a9a:cd3a:bc3b:2bf9:be35(Preferido)
Dirección IPv6 temporal. . . . . : 2800:200:e800:3a9a:ac15:d6e4:2df8:befc(Preferido)
Vínculo: dirección IPv6 local. . . . . : fe80::14ff:9d39:7177:3283%18(Preferido)
Dirección IPv4. . . . . : 192.168.0.8(Preferido)
Máscara de subred. . . . . : 255.255.255.0
Concesión obtenida. . . . . : lunes, 4 de mayo de 2026 09:20:12
La concesión expira. . . . . : martes, 5 de mayo de 2026 09:20:12
Puerta de enlace predeterminada. . . . . : fe80::a239:eeff:fe96:f6fc%18
192.168.0.1

Servidor DHCP. . . . . : 192.168.0.1
IAID DHCPv6. . . . . : 218816687
DUID de cliente DHCPv6. . . . . : 00-01-00-01-2C-B5-45-11-0A-E0-AF-DC-0A-BF
Servidores DNS. . . . . : 2800:200:2000::d0
2800:200:2000::c7
190.113.220.18
190.113.220.51
190.113.220.54
2800:200:2000::410
2800:200:2000::d0
2800:200:2000::c7

NetBIOS sobre TCP/IP. . . . . : habilitado

Adaptador de Ethernet Ethernet 2:

Sufijo DNS específico para la conexión. . :
Descripción. . . . . : VirtualBox Host-Only Ethernet Adapter
Dirección física. . . . . : 0A-00-27-00-00-0C
DHCP habilitado. . . . . : no
Configuración automática habilitada. . . : sí
Vínculo: dirección IPv6 local. . . . . : fe80::2294:77e8:a943:f67e%12(Preferido)
Dirección IPv4. . . . . : 192.168.56.1(Preferido)
Máscara de subred. . . . . : 255.255.255.0
Puerta de enlace predeterminada. . . . . :
IAID DHCPv6. . . . . : 772407335
DUID de cliente DHCPv6. . . . . : 00-01-00-01-2C-B5-45-11-0A-E0-AF-DC-0A-BF
Servidores DNS. . . . . : fec0:0:0:ffff::1%1
fec0:0:0:ffff::2%1
fec0:0:0:ffff::3%1

NetBIOS sobre TCP/IP. . . . . : habilitado
```


El protocolo Dynamic Host Configuration Protocol fue identificado mediante el comando `ipconfig /all`, observándose que el equipo recibe su dirección IP de forma automática desde el servidor DHCP ubicado en el router (192.168.0.1).

```
DHCP habilitado . . . . . : sí
```

Esto confirma que la asignación de direcciones en la red del sistema IoT es dinámica. La dirección IPv4 192.168.0.8 fue asignada automáticamente junto con parámetros como máscara de subred, puerta de enlace y servidores DNS. Además, se observó un período de concesión (lease time), lo que indica que las direcciones IP son renovadas periódicamente, permitiendo flexibilidad y escalabilidad en la red del sistema.

```
Concesión obtenida. . . . . : lunes, 4 de mayo de 2026 09:20:12
La concesión expira . . . . . : martes, 5 de mayo de 2026 09:20:12
```

Protocolo DHCP en la raspberry pi zero 2w

```
pi@raspberrypi:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 88:a2:9e:5a:fc:5e brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.12/24 brd 192.168.0.255 scope global dynamic noprefixroute wlan0
        valid_lft 84727sec preferred_lft 84727sec
    inet6 2800:200:e800:3a9a:8aa2:9eff:fe5a:fc5e/64 scope global dynamic mngtmpaddr proto kernel_r
        valid_lft 15551999sec preferred_lft 2591999sec
    inet6 fe80::8aa2:9eff:fe5a:fc5e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

El protocolo Dynamic Host Configuration Protocol fue verificado en la Raspberry Pi mediante el comando `ip a`, observándose que la interfaz inalámbrica wlan0 recibió la dirección IP 192.168.0.12 de forma dinámica.

```
inet 192.168.0.12/24 brd 192.168.0.255 scope global dynamic noprefixroute wlan0
    valid_lft 84727sec preferred_lft 84727sec
```

Esto confirma que la asignación de direcciones en la red del sistema IoT es gestionada automáticamente por el servidor DHCP del router.

Las métricas de red:

1. Latencia (Round-Trip Time - RTT)

La latencia varía dependiendo del protocolo utilizado para medirla (ARP, ICMP o TCP)

- **Nodo Sensor (ESP32-CAM - 192.168.0.10):**
 - **Latencia ICMP (PowerShell):** Promedio de 3.5 ms (Mínima: 2 ms, Máxima: 5 ms).
 - **Latencia ARP/TCP (Nmap):** Registró latencias entre 49 ms y 69 ms en la capa de enlace y transporte.
- **Nodo de Control (Raspberry Pi - 192.168.0.12):**
 - **Latencia ICMP (PowerShell):** Promedio de 38 ms (Mínima: 3 ms, Máxima: 73 ms).
 - **Latencia TCP (Nmap):** En el escaneo de puertos registró un tiempo de respuesta de 5.6 ms.
- **Gateway / Router (192.168.0.1):**
 - Presenta una latencia casi nula, con un promedio de 0.5 ms en ICMP y 5 ms a 7 ms en resoluciones ARP, confirmando una conexión directa y sin congestión hacia la salida de Internet.

2. Jitter (Variación de Retardo):

Mide cuánto fluctúan los tiempos de llegada de los paquetes entre un punto y otro.

El Jitter se puede calcular observando la diferencia entre los tiempos máximos y mínimos de las respuestas del ping en PowerShell.

- **Jitter en la Raspberry Pi:** Presenta una fluctuación considerable de 70 ms (diferencia entre su máximo de 73 ms y su mínimo de 3 ms). Esto es un comportamiento típico y esperado en dispositivos conectados mediante Wi-Fi (802.11n) debido a la contención del medio (CSMA/CA).
- **Jitter en el ESP32:** Presenta una red sumamente estable en ese instante de prueba, con una fluctuación de apenas 3 ms (diferencia entre 5 ms y 2 ms).

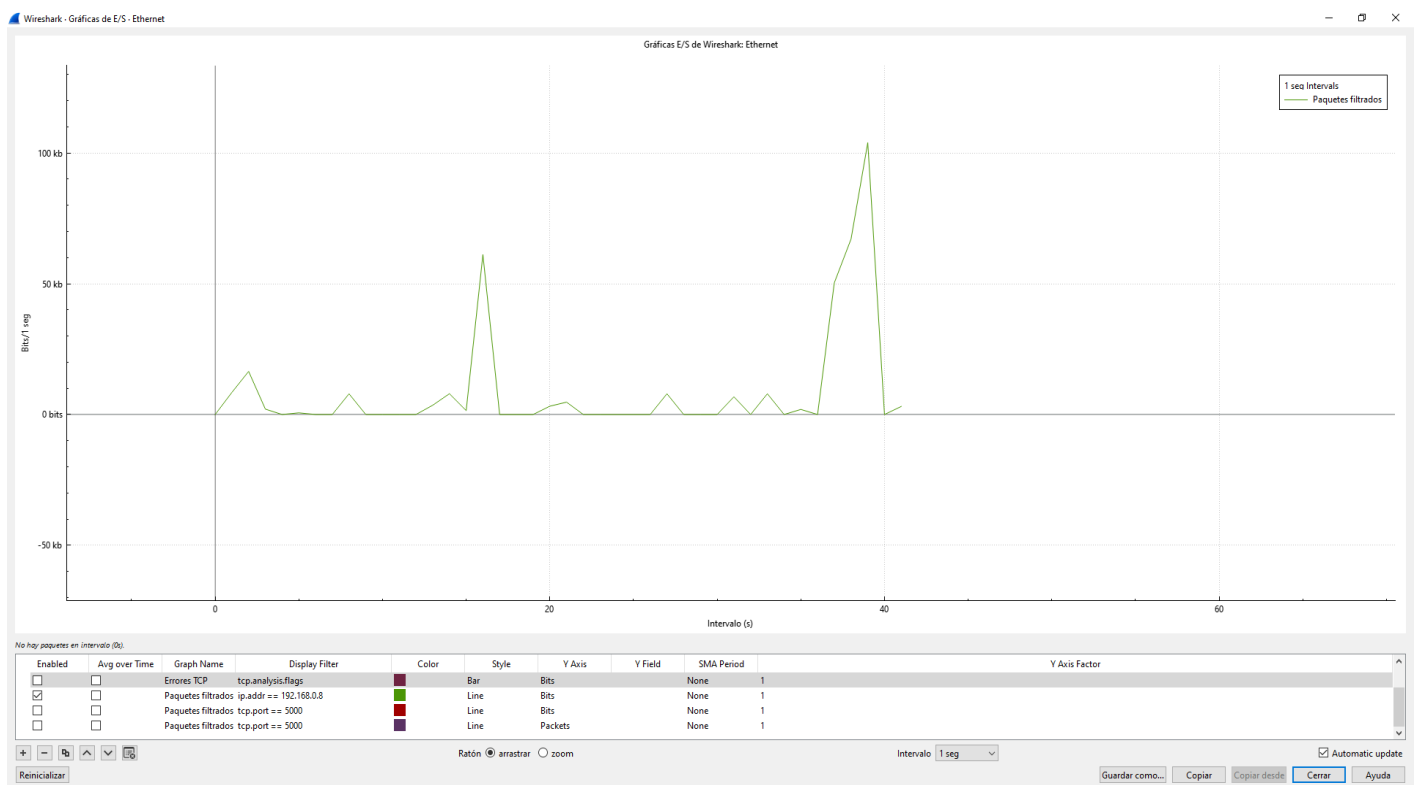
3. Pérdida de Paquetes (Packet Loss) y Disponibilidad

- Basado en la ejecución del script de PowerShell, se enviaron 2 solicitudes ICMP a cada host de la red.
- Tanto la Raspberry Pi, el ESP32 y el Gateway devolvieron resultados de "Respuestas: 2/2", lo que arroja una pérdida de paquetes del 0% en la red local durante el periodo de prueba, garantizando alta disponibilidad a nivel LAN.

4. Throughput (Tasa de Transferencia)

Throughput significa rendimiento efectivo o tasa de transferencia real de datos en una red. Es la cantidad de información útil que se transmite exitosamente por unidad de tiempo, normalmente expresada en bits por segundo (bps), Mbps, o Gbps.

Throughput mínimo (0.746kbs)



- **Throughput de Pico (Carga útil):** Las ráfagas generadas por las peticiones alcanzan aproximadamente los 100 kbps.
- **Throughput de Reposo (Control):** En los momentos de inactividad o de solo intercambio de tramas de control, el rendimiento cae a niveles mínimos registrados de 0.746 kbps, lo que demuestra la eficiencia energética y de ancho de banda del sistema cuando no está capturando imágenes.

5. Tamaño de Ventana TCP (TCP Window Size)

- En la comunicación entre la PC (192.168.0.8) y la Raspberry Pi (192.168.0.12), se observan tamaños de ventana (Win=) de 64128 bytes y hasta 262400 bytes por parte de la PC.

```

226 5001 → 50423 [PSH, ACK] Seq=1 Ack=156 Win=64128 Len=172 [TCP PDU reassembled in 3826]
60 53130 → 5000 [ACK] Seq=4471 Ack=177 Win=64128 Len=0
60 HTTP/1.1 200 OK (text/html)
54 50423 → 5001 [ACK] Seq=156 Ack=176 Win=262400 Len=0
54 50423 → 5001 [FIN, ACK] Seq=156 Ack=176 Win=262400 Len=0
60 5001 → 50423 [ACK] Seq=176 Ack=157 Win=64128 Len=0
54 50024 → 22 [ACK] Seq=1 Ack=273 Win=1025 Len=0

```